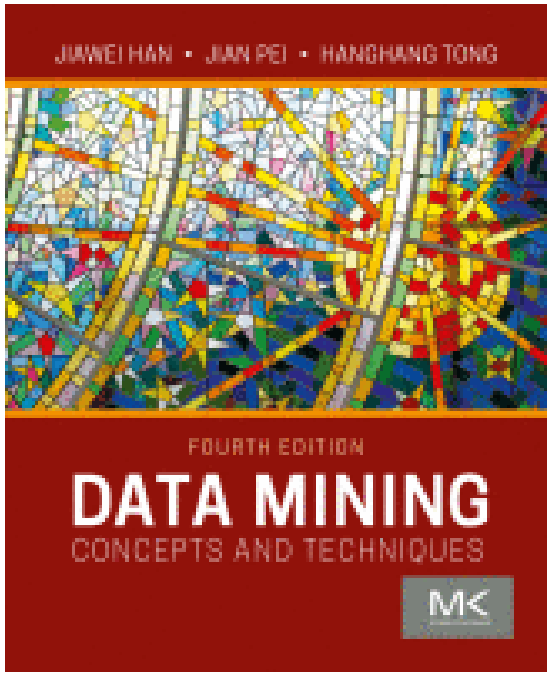
Deep Learning for Tabular Data

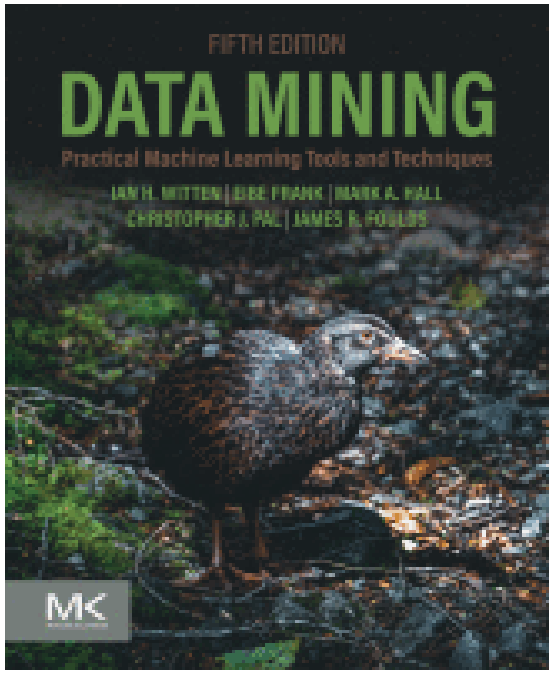
IF5141 dan IF4041 - Penambangan Data

©2026 - Tim Pengajar IF4041 dan IF5141 Semester I TA 2026/2027

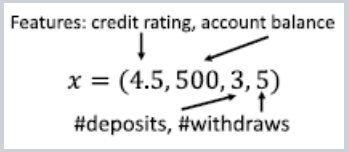
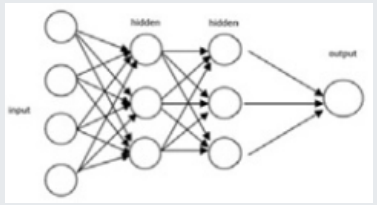
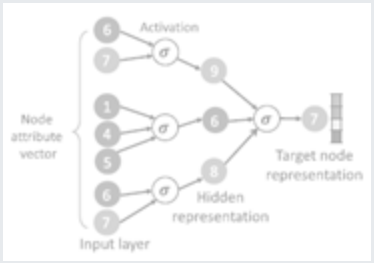
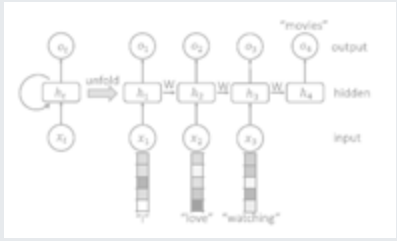
References

- Han, J., Pei, J., & Tong, H. (2024). *Data mining: Concepts and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/C2013-0-18660-6
- Witten, I. H., Hall, M. A., Foulds, J. R., Frank, E., & Pal, C. J. (2025). *Data mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/c2022-0-02092-4





Overview of deep learning architecture

Data Type	*Multidimensional data* (i.e., tabular)	*Relational* data (i.e., graph)	*Sequence* data (i.e., text)	*Grid-like* data (i.e., image)
				
Deep Learning (DL) Architecture	Feed-forward neural networks 	Graph neural networks 	Recurrent neural networks 	Convolutional neural networks 

Feed-forward neural network is primarily designed for *multidimensional* data. That is, each data tuple is represented as an n_0-dimensional vector of attributes $\mathrm{X} = (X_1, \ldots, X_{n_0})$ where n_0 is the number of attributes.

Data Type

Tabular Data

- **Tabular data** is data organized in a **table of rows and columns**, where each row represents an observation and each column represents a feature or attribute.
  - E.g. Adult dataset: Predict whether income exceeds $50K/yr based on census data.

# age	workclass	# fnlwgt	education	# education-...	marital-sta...	occupation	relationship	race	sex
39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male
50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male
38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male
53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male
28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female
37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female
49	Private	160187	9th	5	Married-spouse-absent	Other-service	Not-in-family	Black	Female

https://archive.ics.uci.edu/dataset/20/census+income

Different Approaches to making predictions from data based on Machine Learning

- ***Classical ML* techniques**, which make predictions directly from a set of features that have been prespecified by the user
- E.g. K-Nearest Neighbors Algorithm (KNN)

- ***Representation learning* techniques**, which transform features into some intermediate representation before mapping them to final predictions
- E.g. Principal Component Analysis (PCA) followed by KNN

- ***Deep learning* techniques**, a form of representation learning that uses multiple transformation steps to create very complex features.
- E.g. Deep multilayer perceptrons (Deep MLP)

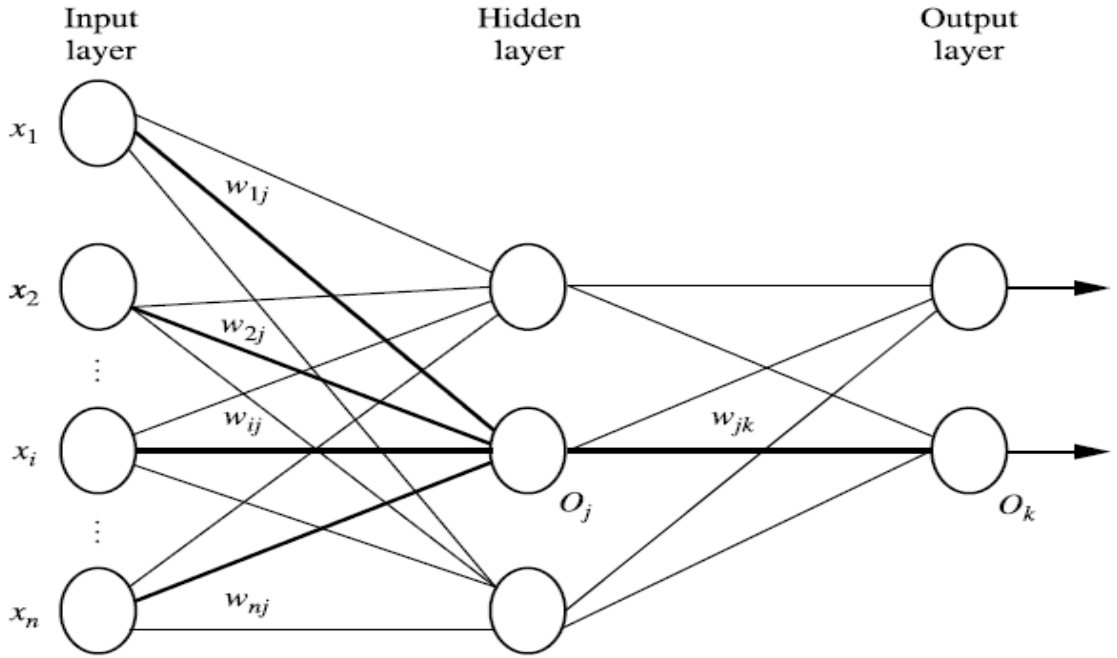
DL Architecture

Deep Learning

- Deep learning aims to automatically learn effective representation (i.e., features, attributes) from the input data to facilitate data mining tasks (e.g., classification, clustering).
- Deep learning is based on **artificial neural networks (ANNs)** (or **neural networks** for short). The advantages of neural networks:
  - able to learn **any** nonlinear function that maps the input data tuple to the output (e.g., the class label), and
  - the **automatic** feature learning.

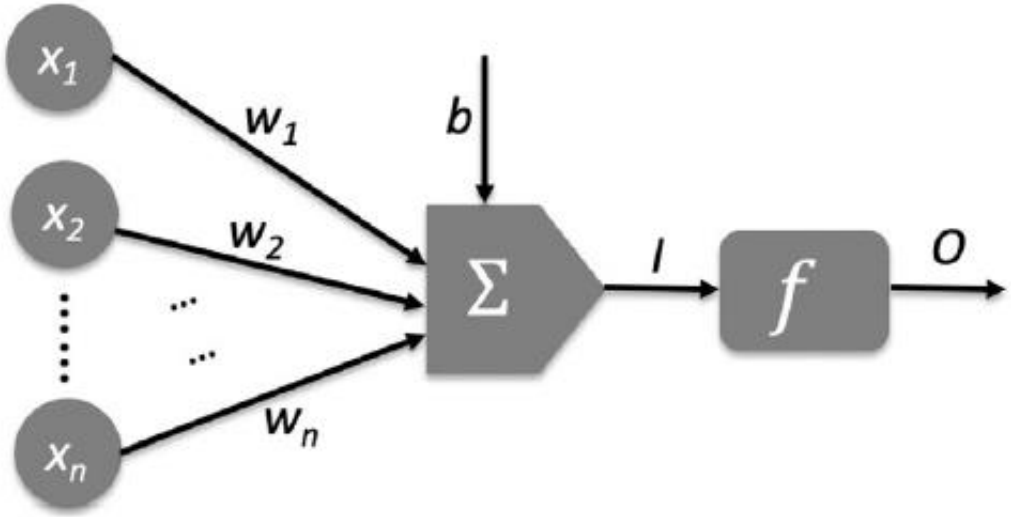
Neural Network

- A **neural network** is a set of connected input-output units in which each connection has a **weight** associated with it.
  - During the learning phase, the network learns by **adjusting the weights** to predict the correct target values (e.g., class labels) of the input tuples.
- Depending on **how different units are organized** with each other; there are many different kinds of neural networks.
  - Among them is a **multilayer feed-forward neural network** (as shown in the figure beside), which consists of an input layer, one (or more) hidden layers, and an output layer.
    - It is a **feed-forward** network since none of the weights cycles back to an input unit or to a previous layer's output unit.



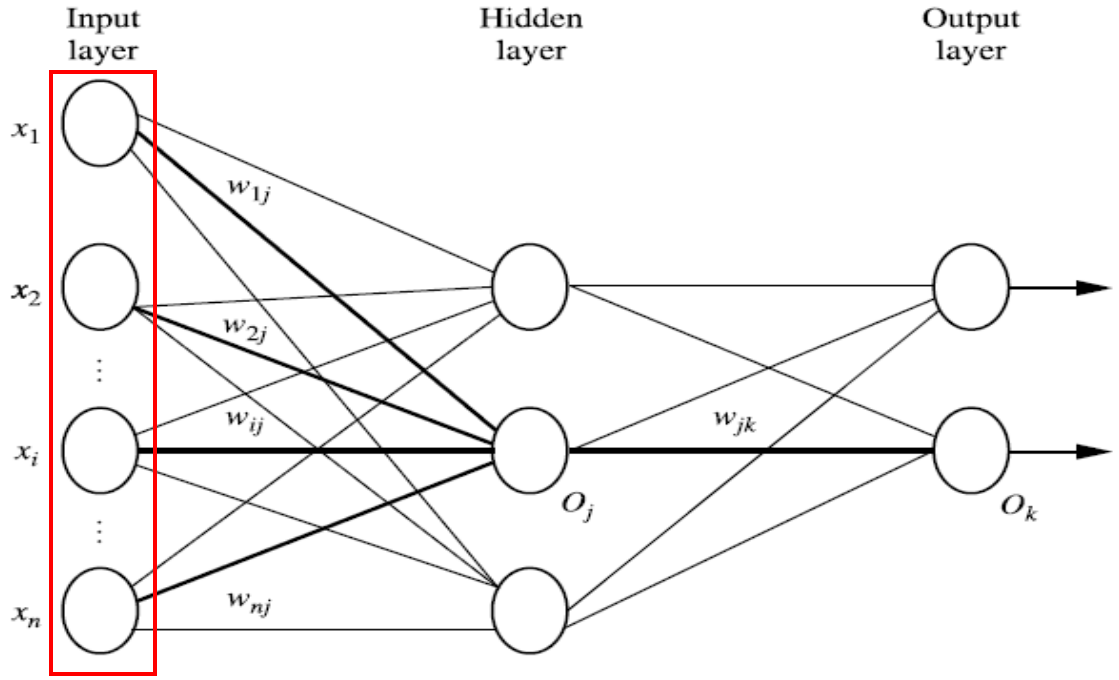
Unit

- A neural network is essentially a collection of interconnected units.
  - **Perceptron** can be viewed as a unit.
  - Figure beside is an illustrative example of unit.
- Given the inputs $X_1, \dots, X_n$, a **unit** takes a linear weighted sum and then passes the sum through an activation function $f(\cdot)$.
  - w_i $(i = 1, \dots, n)$ are weights .
  - b is the bias scalar.
  - $I = \sum_{i=1}^{n} X_i w_i + b$ is the net input of the activation function.
  - $O = f(I)$ is the output of the unit.



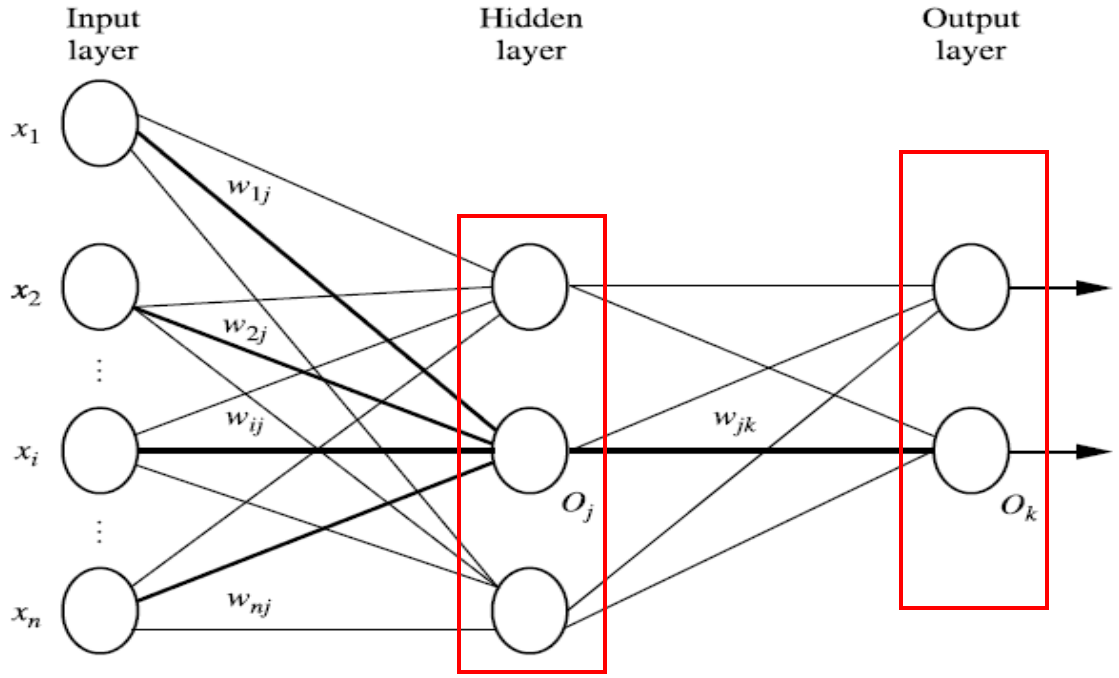
Neural Network – Input Layer

- The units in the input layer are called **input units**.
- The inputs are fed simultaneously into the units making up the **input layer**.
  - The inputs to the network correspond to the attributes measured for each training tuple.
  - These inputs pass through the input layer and are then weighted and fed simultaneously to a second layer of "neuron-like" units, known as a **hidden layer**.



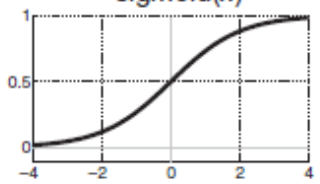
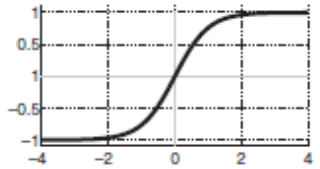
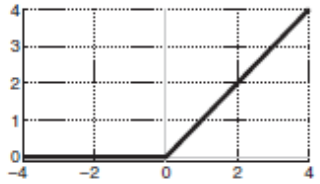
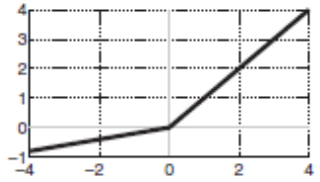
Neural Network – Hidden and Output Layer

- The units in the hidden layers and output layer are referred to as **neurons** or as **output units**.
  - In this example, it has **two layers of output units**. Therefore, it is a **two-layer** neural network.
- A second layer of "neuron-like" units is known as a **hidden layer**.
  - The number of hidden layers could be arbitrary (1 hidden layer in this example).
    - The outputs of the hidden layer units can be input to another hidden layer, and so on.
    - Outputs of the last hidden layer are input to units making up the **output layer**.
- The **output layer** emits the network's predictions for given tuples.



Activation Function

- Each unit in the hidden and output layers takes its net input and then applies an **activation function** to it.
  - **Sigmoid**
    - A widely used activation function.
    - Outputs values between **0 and 1**.
  - **Tanh**
    - Similar to sigmoid, but its output is centered around **0**.
    - Sometimes preferred over sigmoid.
  - **ReLU (Rectify)**
    - A popular choice for **hidden layers** in deep learning.
    - Outputs **0 for negative inputs** and the input value for positive inputs.
    - Helps reduce the **vanishing/exploding gradient problem**.
  - **Piecewise Linear**
    - Similar to ReLU, but provides a non-zero gradient for negative inputs.
    - Increasingly popular in deep learning systems.

Name and Graph	Function
sigmoid(x) 	$h(x) = \frac{1}{1 + \exp(-x)}$
tanh(x) 	$h(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$
rectify(x) 	$h(x) = \max(0, x)$
pw_linear(x) 	$h(x) = \begin{cases} x \text{ if } x \geq 0 \\ ax \text{ if } x < 0 \end{cases}$

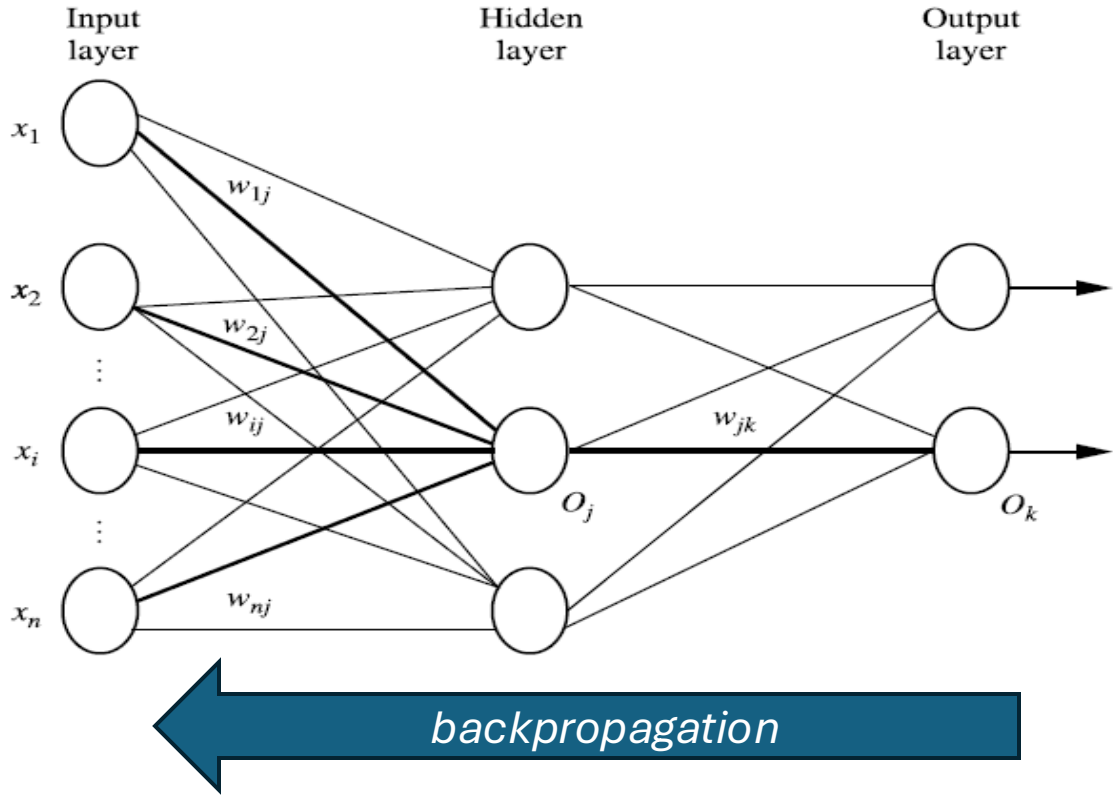
Loss Function

- Deep learning formulates the underlying problem as a network architecture whose output layer defines a loss function needed for learning.
  - The loss function is computed in the final layer.
- The choice of loss function depends on the type of prediction:
  - Squared error → continuous prediction
  - Cross entropy → binary classification
  - Categorical cross entropy → multiclass classification

Loss Name, $L(f_i(\mathbf{x}_i;\theta), \mathbf{y}_i)=$
Squared error, $\sum_{k=1}^{K}(f_k(\mathbf{x}) - y_k)^2$
Cross entropy, $-\sum_{k=1}^{K}[y_k \log f_k(\mathbf{x}) + (1 - y_k)\log(1 - f_k(\mathbf{x}))]$
Categorical cross entropy, $-\sum_{k=1}^{K} y_k \log f_k(\mathbf{x})$

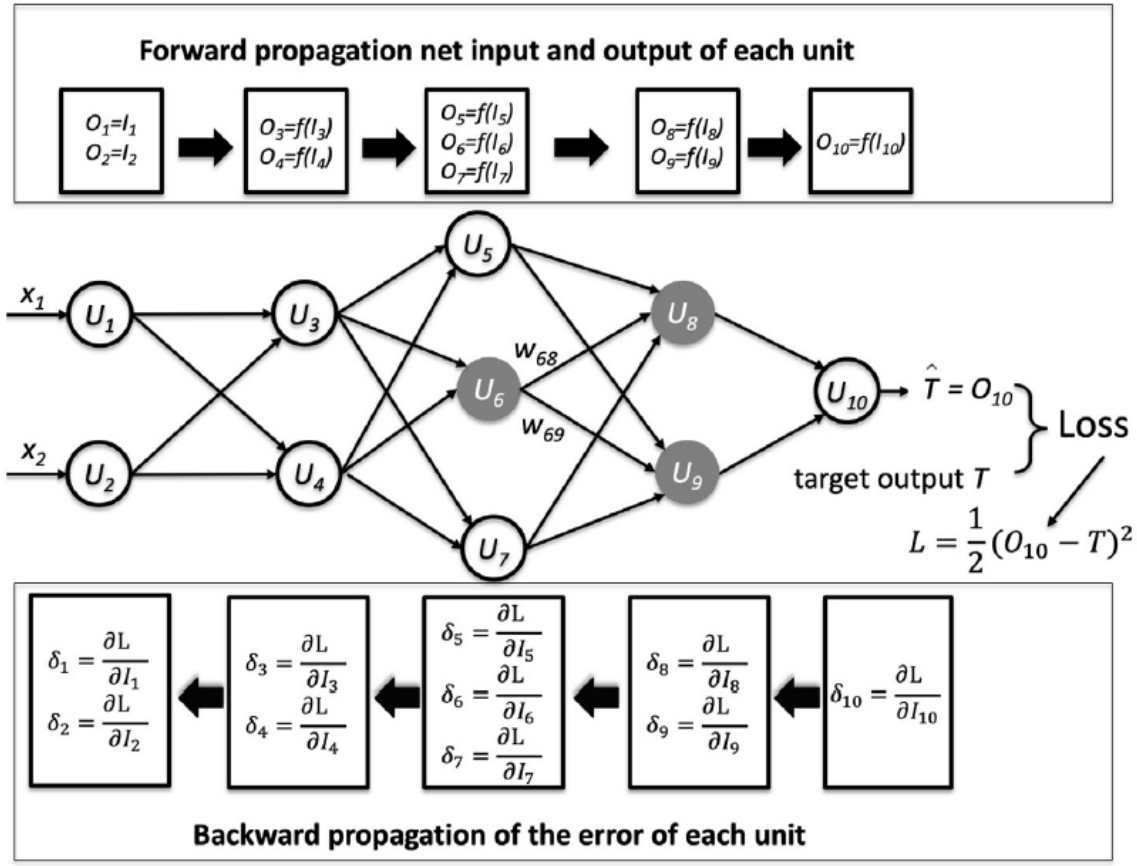
Backpropagation algorithm

- The core algorithmic framework to train a deep neural network is the **backpropagation** algorithm.
- For each training tuple, the weights as well as the bias values are modified to minimize the difference between the network's prediction and the actual target value.
  - These modifications are made in the "backward" direction (i.e., from the output layer) through each hidden layer down to the first hidden layer (hence the name *backpropagation*).



An illustration of the backpropagation algorithm

- Given a training tuple, the algorithm first forward propagates the net input and output of each unit (the upper part of the figure).
- Then, it backward propagates the error of each unit (the bottom part of the figure).
- The goal of backpropagation is to adjust the weights and bias scalars so that the predicted output matches the actual target output.
  - That is, to minimize the loss.



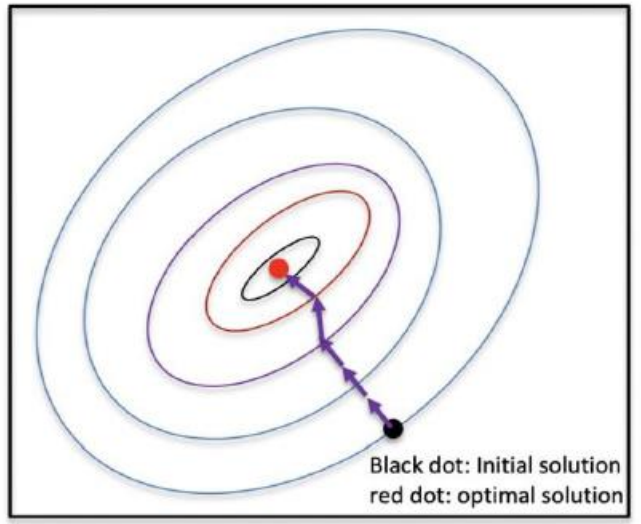
Epoch and Batch

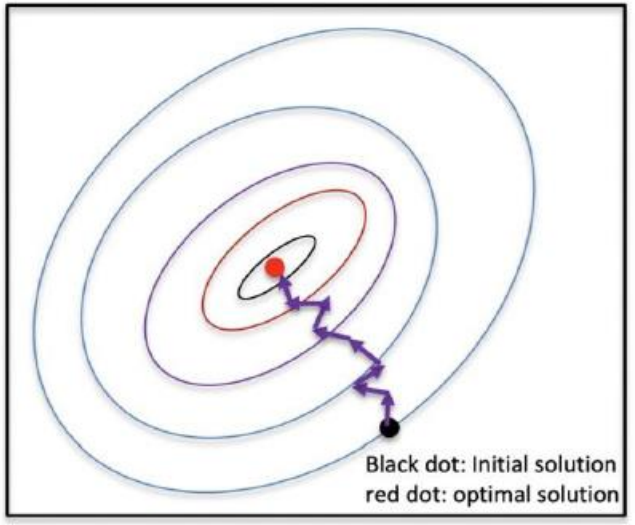
- **Epoch:** One iteration of the training process (in which a set of training tuples is used to update the model's weights and biases).
- **Batch:** A set of training tuples used to update the model.
  - **Full batch:** Uses **all training tuples** in an epoch.
  - **Mini-batch:** Uses **a small subset of training tuples** in an epoch.
- For example, suppose we have **1,000 training samples**:
  - **Full batch:**
    - 1 epoch contains 1,000 samples → 1 epoch is needed to process all 1,000 samples.
  - **Mini-batch = 100:**
    - 1 epoch contains 100 samples → 10 epochs are needed to process all 1,000 samples.

Epoch and Batch (cont.)

	Gradient Descent	Stochastic Gradient Descent
Data for Weight Update	Uses **all training data (full batch)** to calculate the gradient in each epoch.	Uses only a **small mini-batch** of training data in each epoch.
Epochs	Typically requires **fewer epochs** to reach the optimal solution.	Typically requires **more epochs** to reach the optimal solution.
Arrow Direction (in figure)	Finds a more accurate direction toward minimizing the loss.	Estimates the direction toward minimizing the loss.
Overall efficiency	However, each epoch is **more computationally expensive**.	Each epoch is **faster and computationally more efficient**.

Gradient Descent uses more data per update, while SGD uses a mini-batch to make faster updates.

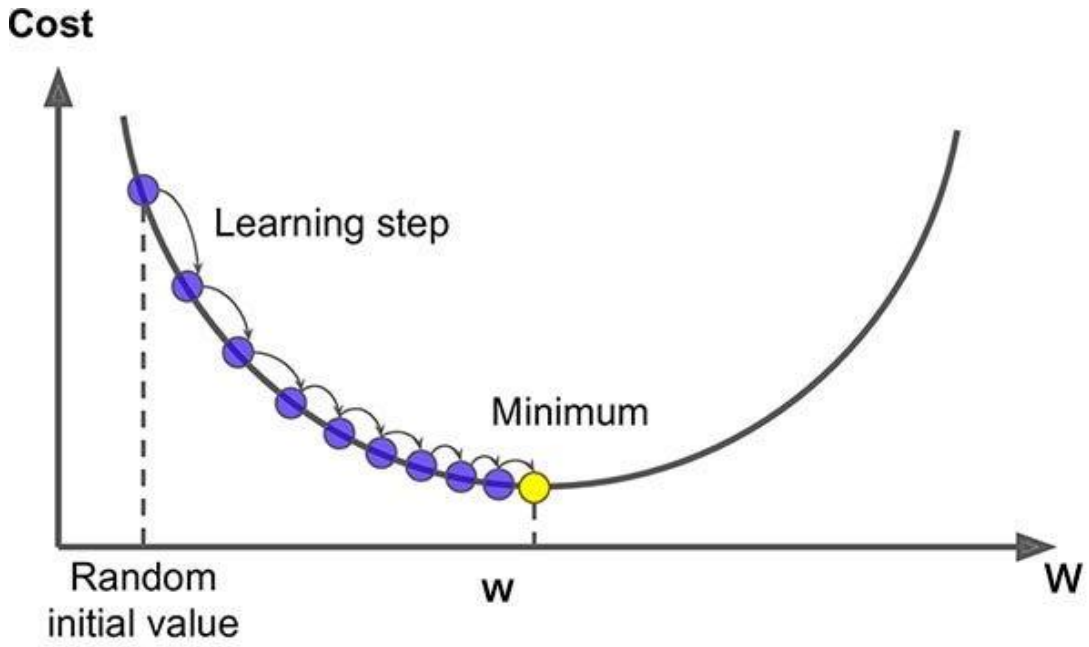




(b) Stochastic gradient descent

Gradient Descent

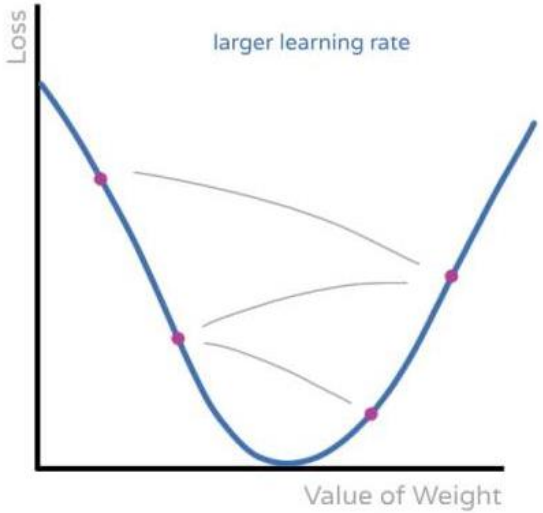
- Backpropagation uses using a **gradient descent** method to search for a set of weights that fit the training data to minimize the loss between the network's prediction and the known target value of the tuples.
- A *gradient descent* strategy performs greedy hill-descending:
  - To start with, the weights are initialized to random probability values.
  - At each iteration or step along the way, the algorithm moves toward what appears to be the best solution at the moment, without backtracking.
    - The weights are updated at each iteration.
  - Eventually, they converge to a local optimum solution.



Learning Rate

- **Learning rate** is a constant typically having a value between 0.0 and 1.0.
  - It is used in **gradient descent** to minimize the loss.
  - It controls how much the model's weights are updated during training.
- A suitable learning rate helps the model avoid getting stuck in a **local minimum** and find a high-quality solution.
  - **Too small:** learning is very slow.
  - **Too large:** the model may change back and forth between inadequate solutions.
- A common rule of thumb is to set the learning rate to **1/t**, where *t* is the number of training iterations.





Task

Use Case Description

- Dataset contains demographic information from **32k adults** collected by the **US Census**.
  - https://archive.ics.uci.edu/dataset/20/census+income
- Goal: predict whether a person's **income is above $50K**.
- Build an **ANN model for binary classification**.

Data Preparation

- **14 features** and one **binary target (Income)**.
- Features include:
  - Numerical features
  - Categorical features
- Categorical features are converted to numbers using **numerical encoding**.
- The original training set is split into:
  - **70% training**
  - **30% validation**
- The original **testing set is used as the testing set**.
- All features are **normalized to [0, 1]**.

Exercise: Adult Data Classification with ANN

Build a network with 14-6-6-1 units

- Layer 1 (input): Dense Layer, Shape 14
- Layer 2 (hidden): Dense Layer, Shape 6, **ReLU activation**
- Layer 3 (hidden): Dense Layer, Shape 6, **ReLU activation**
- Layer 4 (output): Dense Layer, Shape 1, **Sigmoid activation**

Train the network

- Training and validation sets are provided.
- Input: 14 features, excluding the **Income** target.
- Target: **Income**.
- Loss function: **Binary Cross Entropy**.
- Batch size: **100**
- Training: **20 epochs**
- Optimizer: **Adam**

Apply the trained network to the testing set

- Testing sets are provided.
- All 14 features are used as inputs.
- Output: predicted probability.

Convert the network output to binary categories

- If predicted probability **> 0.5** → category **>50K**.
- Otherwise → category **≤50K**.

Assess the accuracy

- Evaluate the model's **accuracy**.

Build a network with 14-6-6-1 units

- Layer 1 (input): Dense Layer, Shape 14
- Layer 2 (hidden): Dense Layer, Shape 6, **ReLU activation**
- Layer 3 (hidden): Dense Layer, Shape 6, **ReLU activation**
- Layer 4 (output): Dense Layer, Shape 1, **Sigmoid activation**

```
# Import TensorFlow untuk membangun dan melatih neural network.
import tensorflow as tf

# Membuat Sequential model yang terdiri dari beberapa layer secara berurutan.
model = tf.keras.Sequential()

# Menentukan input layer dengan 14 input features.
model.add(tf.keras.layers.Input(shape=(14,)))

# Menambahkan hidden layer pertama dengan 6 neurons dan ReLU activation.
model.add(tf.keras.layers.Dense(6, activation="relu"))

# Menambahkan hidden layer kedua dengan 6 neurons dan ReLU activation.
model.add(tf.keras.layers.Dense(6, activation="relu"))

# Menambahkan output layer dengan 1 neuron dan sigmoid activation.
model.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

Train the network

- Training and validation sets are provided.
- Input: 14 features, excluding the **Income** target.
- Target: **Income**.
- Loss function: **Binary Cross Entropy**.
- Batch size: **100**
- Training: **20 epochs**
- Optimizer: **Adam**

```
# Menentukan Adam sebagai optimizer dan Binary Cross Entropy sebagai loss function.
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
# Melatih neural network menggunakan training set.
history = model.fit(
    X_train,                # Training features.
    y_train,                # Training target.
    validation_data=(X_val, y_val),  # Validation set untuk memantau performa selama training.
    batch_size=100,         # Menggunakan 100 training tuples dalam setiap mini-batch.
    epochs=20,              # Melakukan training selama 20 epochs.
    verbose=1               # Menampilkan progress training.
)
```

What is the role of the “Adam optimizer”?

- **Adam (Adaptive Moment Estimation)** is a commonly used momentum-based approach with an **adaptive learning rate** in deep learning.
- **Momentum-based approaches** consider **historical gradients** when updating the model parameters.
  - Standard gradient descent uses the **current gradient** to update the model parameters.
- Compared with standard stochastic gradient descent, momentum-based approaches can **reduce the variance of the gradient estimated from the mini-batch**.

Han, et al (2024)

Apply the trained network to the testing set

- Testing sets are provided.
- All 14 features are used as inputs.
- Output: predicted probability.

```
# Menghasilkan predicted probability untuk setiap data pada testing set.
y_pred_probability = model.predict(X_test)

# Mengubah hasil prediksi dari bentuk (n, 1) menjadi array satu dimensi.
y_pred_probability = y_pred_probability.ravel()

# Menampilkan beberapa predicted probability pertama.
print(y_pred_probability[:10])
```

```
471/471 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
[0.00203802 0.3269529  0.48637527 0.7922486  0.0043461  0.7654095
 0.02485744 0.0115047  0.7765278  0.69994545]
```

Convert probability into binary categories

- If predicted probability **> 0.5** → category **>50K**. ➔ 1
- Otherwise → category **≤50K ➔ 0**

```
# Mengubah predicted probability menjadi binary category.
# Probability > 0.5 dikategorikan sebagai >50K.
# Probability <= 0.5 dikategorikan sebagai <=50K.
y_pred = (y_pred_probability > 0.5).astype(int)

# Menampilkan beberapa hasil prediksi pertama.
print(y_pred[:10])
```

```
[0 0 0 1 0 1 0 0 1 1]
```

Assess the accuracy

- Evaluate the model's **accuracy**.

```python
# Import accuracy_score untuk menghitung accuracy.
from sklearn.metrics import accuracy_score

# Menghitung accuracy dengan membandingkan actual category dan predicted category.
accuracy = accuracy_score(y_test, y_pred)

# Menampilkan accuracy dalam bentuk persentase.
print(f"Testing Accuracy: {accuracy * 100:.2f}%")
```

```
Testing Accuracy: 78.30%
```

- Full code: https://colab.research.google.com/drive/19AucOfezBurE3OS8FIdr_Bdw5LbNMEeb?usp=sharing

Hyperparameter Tuning

Hyperparameter Tuning

- Hyperparameters are tuned by identifying what settings lead to the best performance on the validation set.
- Common hyperparameters include:
  - the number of **hidden units** and **layers** (and their connectivity), the form of **activation functions**, the weight used for the **regularization** term, **parameters** of the learning algorithm itself, etc.
- Hyperparameter tuning techniques:
  - heuristically by hand (**manual search**): hyperparameters are adjusted manually based on experience and experimental results;
  - using **grid search**: evaluates **every possible** combination of predefined hyperparameter values to find the best configuration;
  - **random search**: where instead of placing a regular grid over the hyperparameter space, probability distributions are specified from which **samples are taken**;
  - using **Bayesian optimization** techniques: to **infer the next** hyperparameter configuration to try (in a sequence of experimental runs).